SCTR'S PUNE INSTITUTE OF COMPUTER TECHNOLOGY

PUNE, MAHARASHTRA

UNIVERSITY HIGHER EDUCATION ERP SYSTEM

GROUP 2 · ACADEMIC & COURSE MODULE

PROJECT STUDY AND IMPLEMENTATION REPORT

Course	Database Management Systems — Major Project
Batch	M7 BATCH
Department	Electronics and Telecommunication Engineering (E&TC)
Prepared for	Group 2
Prepared by	Group members: _____
Evidence cut-off	17 October 2026

Repository-grounded · synthetic UI responses explicitly labelled · no live DB verification claimed

Contents

- 1. Executive Summary
- 2. Project Context, Problem and Objectives
- 3. Architecture and Technology
- 4. Requirements and User Roles
- 5. Group 2 Module Design and Workflows
- 6. Data Model and ER Diagram
- 7. Data Dictionary, Keys and Integrity
- 8. Database Procedures, Triggers and Transactions
- 9. API Surface and Security
- 10. User Interface Evidence
- 11. Cross-Module Integration
- 12. Testing and Evidence Status
- 13. Limitations and Recommendations
- 14. Conclusion
- Appendix A. Recommended Demonstration Test Matrix
- Appendix B. Repository Evidence and References

Reading the evidence labels

Repository facts are attributed to specific source files. Screenshots are authentic React UI captures using illustrative synthetic API responses. Database/API behavior not exercised here is identified as unverified.

1. Executive Summary

The University Higher Education ERP System is a database-centric, seven-module project built around a shared relational schema. Group 2 (Academic & Course) provides the academic reference layer: departments, programs, batches, sections, subjects, curriculum mapping, course offerings, enrolments, faculty-subject allocations, timetabling and the academic calendar. The module is important because other areas attach their work to the course offering: attendance sessions, examination registrations, enrolments and faculty teaching assignments.

The repository implements the module through React 18 and TypeScript screens, an Express/TypeScript API, and MySQL 8 / MariaDB-compatible SQL. It puts enrolment capacity, duplicate prevention and timetable validation in database constraints, a transaction procedure and triggers, rather than trusting only the browser. This report describes the repository as inspected and separates documented project claims from evidence actually available during preparation.

> Evidence note: The six interface figures in this report are authentic browser captures of the repository React application. The API and database were unavailable in the report sandbox, so Playwright intercepted /api/v1 requests and supplied explicitly synthetic illustrative responses. These are not live-data screenshots and do not verify backend or database behaviour.

At a glance	Repository-backed description
Group 2 table inventory	13 academic and course tables listed in PROJECT_REPORT.md §21 and MODULE_INTEGRATION.md §3
Primary UI pages	Academic masters, Subjects, Course offerings, Timetable and Academic calendar
Key DBMS logic	sp_bulk_enroll_students; enrolment guard/counter triggers; timetable conflict trigger
Repository scale (all modules)	Documentation reports 61 tables, 20 views, 17 procedures, 13 functions, 28 triggers and 138 foreign keys
Current year shown in fixtures	A.Y. 2026–27, Semester 5, matching the repository seed flags; not freshly queried from a database

2. Project Context, Problem and Objectives

2.1 Problem statement

Academic data is often spread across department spreadsheets and separate systems. That makes it difficult to keep curriculum definitions, semester offerings, section membership, faculty assignments and room schedules consistent. In a joined ERP, one course-offering record can be referenced by enrolment, attendance, examinations and timetable functions, reducing duplicated subject or faculty data.

2.2 Group 2 objectives

- Maintain a single department/program/batch/section structure for the rest of the ERP to reference.
- Maintain subject metadata and map subjects into each program and semester through program_subjects.
- Create semester- and section-specific course offerings that connect subjects, programs, faculty and capacity.
- Enrol students individually or in bulk while enforcing duplicate and capacity rules.
- Schedule classes by day, start/end time and room, and reject defined faculty, room and section clashes.
- Publish term dates, holidays, examinations and institute events through academic_calendar.
- Provide authenticated, role-aware pages and administrator-controlled write operations.

2.3 Scope boundaries

In scope for Group 2	Related but owned elsewhere
Departments, programs, batches, sections, years, semesters, subjects, program-subject mappings, course offerings, enrolments, faculty-subject mapping, timetable, academic calendar.	Student identity/admission, faculty HR/payroll, attendance marking, examination schedules/marks/results, fee collection and hostel records. These consume shared academic keys but are not Group 2 CRUD domains.

3. Architecture and Technology

The repository uses a layered browser-to-database path. The React client calls relative /api/v1 URLs; Vite proxies /api to the Express service in development. Express routes apply authentication, role checks and Zod request validation, then call controllers, services and parameterised-SQL repositories. The backend uses mysql2/promise to reach the university_erp schema. The project documents MySQL 8 / MariaDB 10.6+ compatibility; a MySQL/MariaDB service was not available in the report-building sandbox.

Layer	Technology / role
Frontend	React 18 · TypeScript · Vite · Tailwind CSS · React Router · Lucide icons · Recharts
Development proxy	Vite /api proxy; browser code uses relative API URLs
Backend	Node 20 · Express · TypeScript (strict) · Zod validation · mysql2/promise
Authentication and access	JWT authentication; role middleware on protected routes; administrator-only writes for Group 2
Database	MySQL 8 / MariaDB 10.6+ · InnoDB · FK/unique/check constraints · transactions · procedures · triggers · views
Documentation artifacts	Generated Markdown data dictionary, Mermaid source and SVG ER diagrams

3.1 Request path

```
Browser (React) → relative /api/v1 request → Vite /api proxy → Express route
→ authenticate / role / Zod validation → controller → academic service
→ repository with bound SQL parameters → MySQL 8 / MariaDB
```

This separation keeps business rules out of React and keeps SQL in repository/data-access code. The bulk-enrolment service calls the stored procedure; the timetable service delegates conflict refusal to the database trigger. UI permissions improve navigation, while the API role checks are the security boundary.

4. Requirements and User Roles

4.1 Functional requirements

ID	Requirement	Primary implementation
FR-01	Maintain departments, programs, batches, sections, years and semesters.	AcademicsMastersPage; /academics/* reference endpoints; relational master tables.
FR-02	Maintain subject catalog with credits, type and contact hours.	SubjectsPage; subjects table; subject-code uniqueness.
FR-03	Map subjects to programs/semesters and define delivered offerings.	program_subjects schema bridge and course_offerings API. No curriculum route was found in the inspected academic.routes.ts; a route/UI curriculum workflow is not verified.
FR-04	Enroll a student in an offering and bulk-enrol a program/semester cohort.	POST /enrollments; POST /enrollments/bulk; sp_bulk_enroll_students.
FR-05	Schedule course slots and expose conflict information.	TimetablePage; timetable table; trigger and GET /timetable/conflicts.
FR-06	Maintain events and holidays by academic year and optional semester.	CalendarPage; academic_calendar table and endpoints.
FR-07	Audit administrator create/update/delete actions.	Academic service audit calls; audit_logs table (as documented).
FR-08	Expose shared offerings to attendance, examination and faculty workflows.	FK-based integration through course_offerings and shared student/faculty masters.

4.2 User access model

Actor / role	Group 2 use (as wired in routes)
ADMIN	May create/update academic subjects and offerings, create sections, set class teacher, enrol/drop, assign faculty-subjects, add/remove timetable slots and calendar events.
FACULTY	Authenticated reads for academic reference data; faculty timetable and faculty-subject reads are role-scoped.
EXAM_CELL, ACCOUNTANT, HOSTEL_ADMIN, HR	Authenticated read access to selected academic reference lists; some routes explicitly permit these staff roles.
STUDENT	Student-specific timetable endpoint is allowed for the student role; general reference routes require authentication. No Group 2 write route is exposed.

The route file explicitly attaches requireRole('ADMIN') to Group 2 write endpoints. Read access is not identical across endpoints: some require only authentication, some use staff-role lists, and the student timetable route has its own allowed-role list. Verify each route before documenting a broader role guarantee.

5. Group 2 Module Design and Workflows

5.1 Academic masters and curriculum

Departments own programs and subject catalog entries. Batches identify cohorts and their academic-year intake; sections group students and can refer to a class teacher. program_subjects is the intersection table for a program-to-subject many-to-many relationship, with semester_no and is_elective attributes. Its composite unique key prevents repeating the same subject in the same program semester. The repository also contains curriculum service methods, but the inspected academic.routes.ts does not mount a curriculum route; this report treats the table as present and the end-to-end curriculum CRUD path as unverified.

5.2 Course offering as the delivery pivot

A course_offerings row binds a subject to a semester and academic year, program, optional section and optional faculty member, and stores capacity, enrolled_count, offering_type and status. The offering is the operational key used downstream by enrolments, attendance sessions, examination registration and timetable slots. The row therefore represents a course actually delivered to a cohort, not just a curriculum catalog item.

5.3 Enrolment workflow

1. An administrator selects a program, semester and optional section for bulk processing.
2. The API requires authentication and the ADMIN role, then calls the academic service.
3. The service invokes sp_bulk_enroll_students with the program, semester, section and actor.
4. The procedure starts a transaction, counts existing pairs, checks offering capacity and stages new student/offering pairs in a temporary table.
5. It inserts eligible missing pairs, returns enrolled/skipped/message outputs, writes an audit entry and reconciles offering counters.
6. The unique student/offering key and enrolment guard protect against duplicates, cancelled offerings and capacity violations at the database layer.

5.4 Timetable and conflict handling

A timetable row identifies an offering, section and faculty member and records day_of_week, start_time, end_time and room_number. The BEFORE INSERT trigger refuses an end time not later than the start time and signals readable conflicts when the same faculty member, room or section already has a row with the same day and start time. Unique keys also cover each faculty/day/start, room/day/start and section/day/start combination. The inspected trigger checks identical start-time slots; it should not be described as a general interval-overlap detector.

5.5 Academic calendar

The academic calendar holds title, description, event_type, start/end dates, holiday flag and a required academic_year_id with an optional semester_id. The UI displays upcoming events first and lists the full event record below.

6. Data Model and ER Diagram

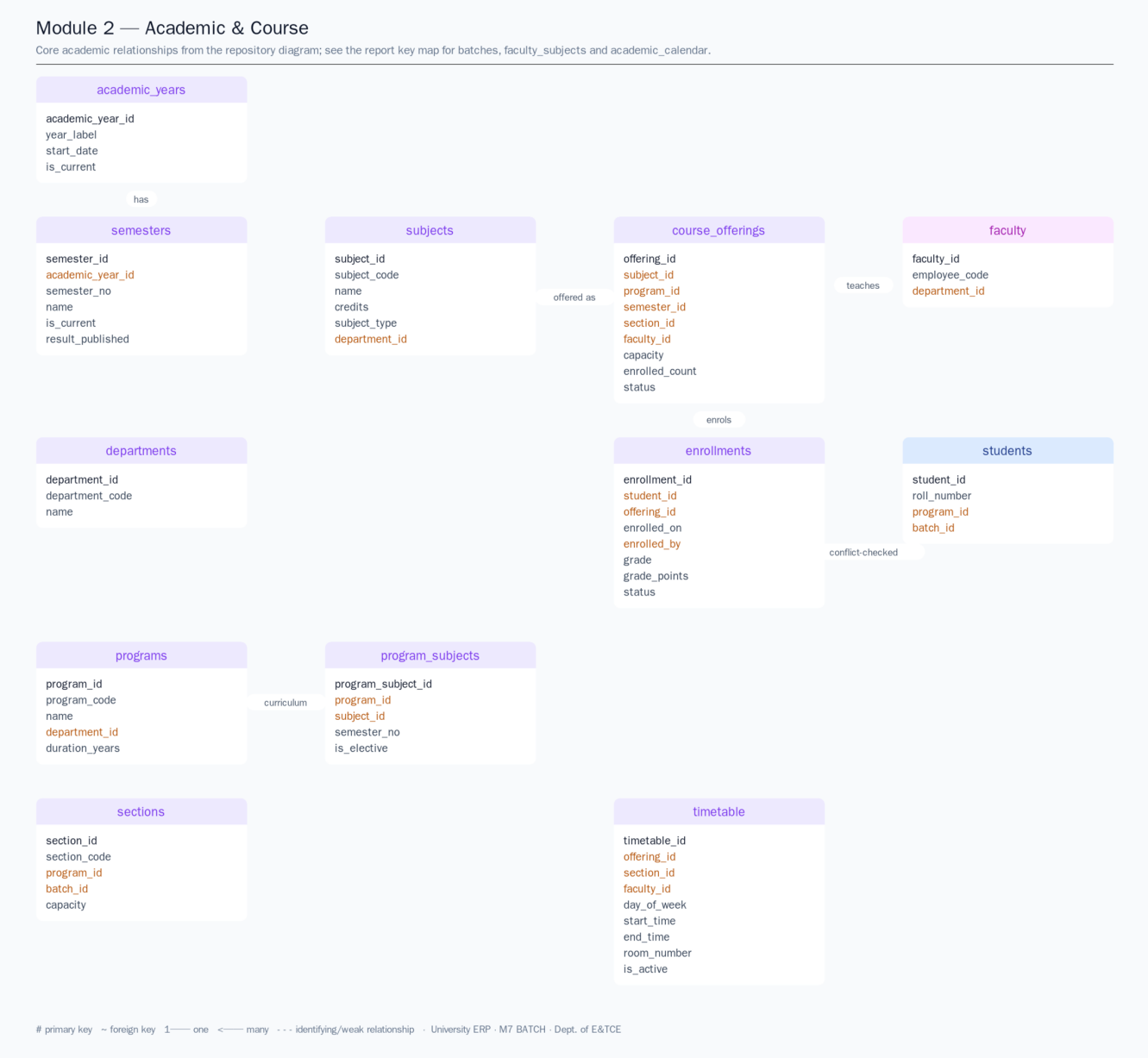


Figure 1. Repository-generated Group 2 core ER diagram. The report shortens the subtitle for print; the original SVG/Mermaid sources remain under docs/er/.

Diagram coverage note: docs/ER_DIAGRAMS.md records 12 entities and 14 relationships for the generated Module 2 SVG. The current Group 2 table inventory in PROJECT_REPORT.md §21 and MODULE_INTEGRATION.md §3 has 13 module tables. The generated picture includes shared students and faculty but omits batches, faculty_subjects and academic_calendar. This report retains the repository diagram and lists the omitted tables and their keys below; treat the graphic as the core diagram, not the complete physical schema.

6.1 Group 2 table inventory

Table	Role in the data model	Primary and selected foreign keys
departments	Shared organizational master.	PK department_id; referenced by programs, subjects and faculty.
programs	Academic program definition.	PK program_id; FK department_id; unique program_code.
batches	Cohort / intake year.	PK batch_id; FK program_id and academic_year_id; unique batch_code.
sections	Teaching group within a program/batch.	PK section_id; FK program_id, batch_id, optional semester_id and class_teacher_id.
subjects	Catalog of taught subject units.	PK subject_id; FK department_id; unique subject_code.
program_subjects	Curriculum mapping bridge.	PK program_subject_id; FKs program_id, subject_id; unique program/subject/semester.
course_offerings	Subject delivered to a term/program/section.	PK offering_id; FKs subject, semester, year, program, optional section/faculty.
enrollments	Student registration against an offering.	PK enrollment_id; FKs student_id and offering_id; unique student/offering.
faculty_subjects	Term-bounded teaching allocation.	PK allocation_id; FKs faculty, subject, academic_year and optional semester.
timetable	Recurring weekly slot.	PK timetable_id; FKs offering, section and faculty.
academic_calendar	Academic-year event/holiday.	PK event_id; FK academic_year_id and optional semester_id.
academic_years	Year master/current-year flag.	PK academic_year_id; unique year_label.
semesters	Term within academic year.	PK semester_id; FK academic_year_id; unique year/semester_no.

6.2 Core relationship map

Parent / master	Child / relationship	Meaning
departments	programs; subjects; faculty	One department may own multiple programs, subjects and faculty records.
programs	batches; sections; program_subjects; course_offerings	Program defines the cohort and curriculum/teaching context.
academic_years	semesters; batches; course_offerings; faculty_subjects; academic_calendar	Year is the temporal parent for term and year-specific records.
semesters	sections; course_offerings; faculty_subjects; academic_calendar	Term context; nullable where the schema allows it.
subjects	program_subjects; course_offerings; faculty_subjects	Catalog subjects map into a curriculum and are assigned/offered.
batches	sections; students	A cohort groups students and its sections.
sections	course_offerings; timetable	Sections receive offerings and weekly slots.
students	enrollments	Student/offering many-to-many association is stored in enrollments.
faculty	course_offerings; faculty_subjects; sections; timetable	Faculty may teach offerings, hold subject allocations, serve as class teacher and appear on slots.
course_offerings	enrollments; timetable	Operational course record is the pivot for enrolment and schedule slots.

7. Data Dictionary, Keys and Integrity

The project data dictionary is generated from the schema and documents key types, nullability, indexes and foreign keys. The excerpts below summarize selected Group 2 constraints; they do not replace the full dictionary or a live SHOW CREATE TABLE review.

Integrity rule	Schema mechanism	Effect
Academic-year identity	UNIQUE academic_years.year_label	Prevents duplicate year labels.
Semester within year	FK semesters.academic_year_id; UNIQUE (academic_year_id, semester_no)	Prevents duplicate semester numbers inside a year.
Program and subject masters	Unique program_code / subject_code; department FKs	Avoids duplicate catalog codes and orphan departments.
Curriculum mapping	UNIQUE (program_id, subject_id, semester_no)	Prevents repeating a subject in a program term.
Section identity	UNIQUE (batch_id, section_code)	Prevents duplicate section labels inside a batch.
Course offering	Unique subject/semester/program/section tuple; FKs to academic masters	Prevents duplicate offering definitions for the same key tuple.
Enrolment identity	UNIQUE (student_id, offering_id)	A student cannot have two enrolment rows for the same offering.
Capacity counter	enrolled_count plus AFTER INSERT/DELETE counter triggers	Keeps the stored offering seat count synchronized with enrolment rows.
Faculty subject allocation	UNIQUE (faculty_id, subject_id, academic_year_id, semester_id)	Prevents duplicate allocation for that faculty, subject and term.
Timetable slot	Unique faculty/day/start, room/day/start and section/day/start keys	Prevents duplicate start slots for the same resource.
Calendar integrity	FK to academic year and optional semester; ENUM event_type	Restricts event associations to existing academic time masters.

7.1 Relational design notes

- program_subjects resolves the many-to-many relationship between programs and subjects while keeping semester number and elective status on the mapping.
- enrollments resolves the many-to-many relationship between students and course offerings and carries enrollment date/status/grade fields.
- course_offerings.enrolled_count is a denormalized counter documented as maintained by insert/delete triggers and the bulk procedure; source rows remain enrollments.
- Foreign keys and unique keys are declared in the schema/constraint scripts and summarized in docs/DATA_DICTIONARY.md.
- The data dictionary describes lifecycle status columns, defaults, timestamp fields and database indexes; refer to the generated dictionary for exact column types.

8. Database Procedures, Triggers and Transactions

DBMS object	Observed responsibility in source
sp_bulk_enroll_students	Starts a transaction; checks existing pairs and capacity; stages missing pairs; inserts enrolments; returns enrolled/skipped/message; handles SQL exceptions with rollback/resignal.
trg_enrollments_bi_guard	Before insert: reads offering state/capacity and student state; refuses missing/cancelled offering, disallowed student state and full offering.
trg_enrollments_ai_count	After insert: recomputes the offering enrolled_count from enrollments.
trg_enrollments_ad_count	After delete: recomputes the offering enrolled_count from enrollments.
trg_timetable_bi_conflict	Before insert: rejects end_time not after start_time, and same day/start-time faculty, room or section clashes.
Unique indexes	Database uniqueness provides a second layer for duplicate enrolment and the three timetable resource/start combinations.

8.1 Why these rules belong in the database

A database guard applies to API calls and direct SQL writes, not just the React forms. The bulk operation has an explicit transaction and error handler, while the timetable trigger raises readable SQLSTATE 45000 errors. The API service converts repository/procedure outcomes into the shared response envelope and audit messages (as documented in the service code).

8.2 Group 2 query/report opportunities

- Offerings by program, semester, subject, faculty and section.
- Seat utilization: enrolled_count divided by capacity, with attention to zero capacity.
- Curriculum completion matrix: program_subjects compared with active offerings.
- Faculty teaching load by year/semester from faculty_subjects and course_offerings.
- Weekly room and section schedules from timetable.
- Calendar events grouped by academic year, semester, type and holiday flag.

9. API Surface and Security

Route group	Methods / examples	Access noted in route file
/academics/departments, /programs, /batches, /years, /semesters, /sections	GET lists/details; current semester lookup.	Authenticated reads.
/academics/subjects	GET list/detail; POST create; PUT update.	Authenticated reads; ADMIN writes.
/academics/offerings	GET list/detail; POST create; PUT update.	Authenticated reads; ADMIN writes.
/academics/enrollments	GET list; POST single; POST /bulk; DELETE by id.	GET restricted to listed staff roles; ADMIN writes.
/academics/faculty-subjects	GET list; POST assign; DELETE allocation.	GET ADMIN/FACULTY; writes ADMIN.
/academics/timetable	GET section/faculty/student; GET conflicts; POST slot; DELETE slot.	Student timetable route permits ADMIN/FACULTY/STUDENT; conflicts ADMIN; slot writes ADMIN.
/academics/calendar	GET list; POST event; DELETE event.	Authenticated read; ADMIN writes.

Route source: backend/src/routes/academic.routes.ts. The base prefix is /api/v1/academics because the router is mounted at /academics. API write routes use authenticate and requireRole('ADMIN'); controller validation and service logic are additional layers. The browser uses relative /api/v1 calls, which is appropriate for the Vite proxy and preview host.

9.1 Security controls described by the repository

- JWT authentication and role middleware on protected endpoints.
- Zod schemas on write endpoints to validate request bodies before service logic.
- Prepared/bound SQL parameters in repository methods rather than concatenated user values.
- Administrator-only API writes for the Group 2 setup and scheduling operations.
- Audit calls on major academic create/update/delete actions, as implemented in academic.service.ts.
- The in-app route/UI restriction is not a substitute for the server-side checks; API authorization is the actual boundary.

10. User Interface Evidence

The following images show the real React application and shared ERP shell, including the available official PICT logo asset. The capture script injects a synthetic ADMIN session token only for local rendering and intercepts API requests with fixed illustrative fixtures. No API mutation was submitted and no database query ran.

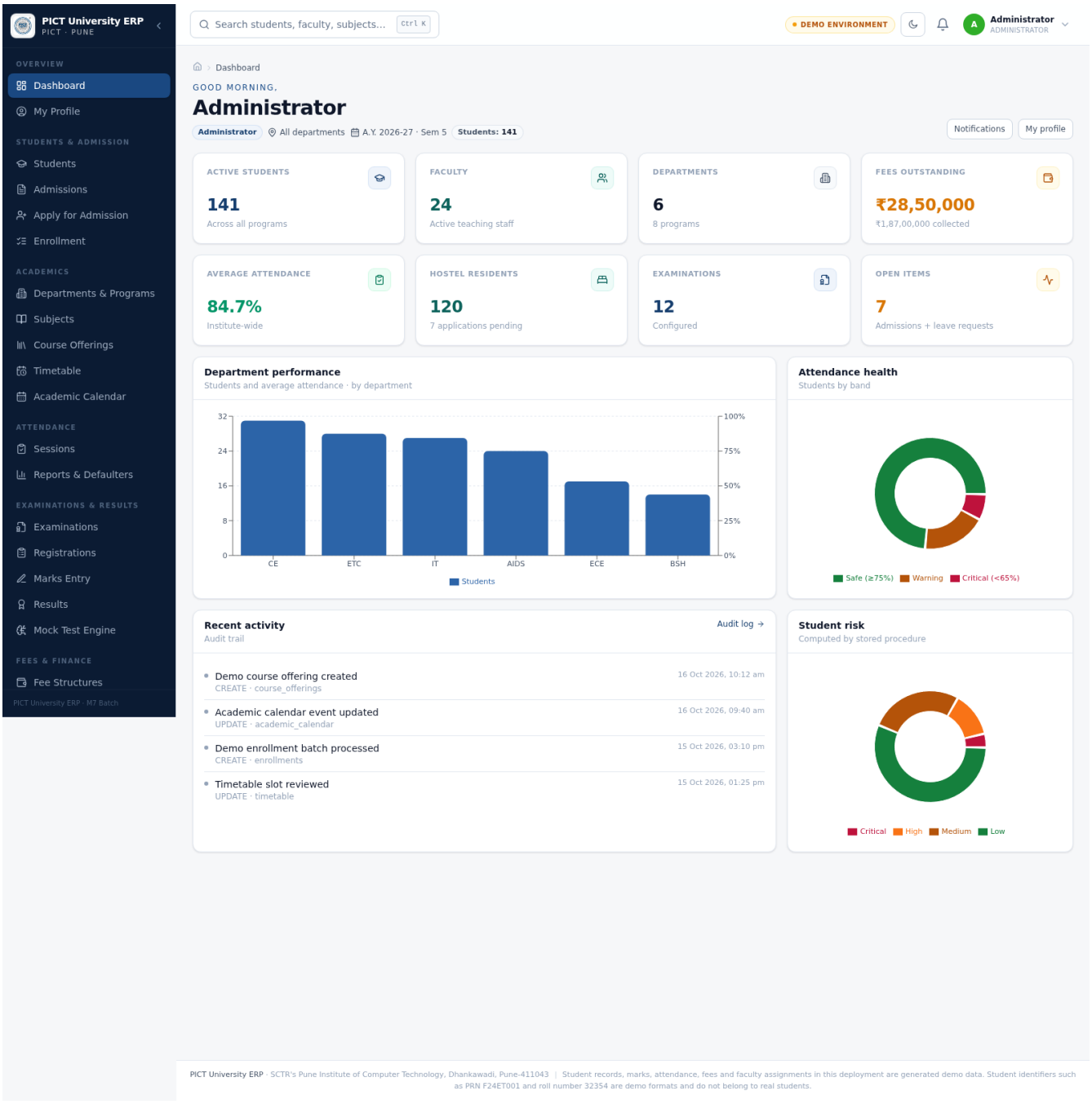


Figure 2. Admin dashboard with illustrative synthetic KPI/chart/activity responses. The 2026–27 / Semester 5 label follows the repository seed; all displayed counts and activities are sample fixture values, not a live dashboard query. The chart's ECE code denotes the separate Electronics and Computer Engineering department (short name E&CE); ETC is the E&TC department code.

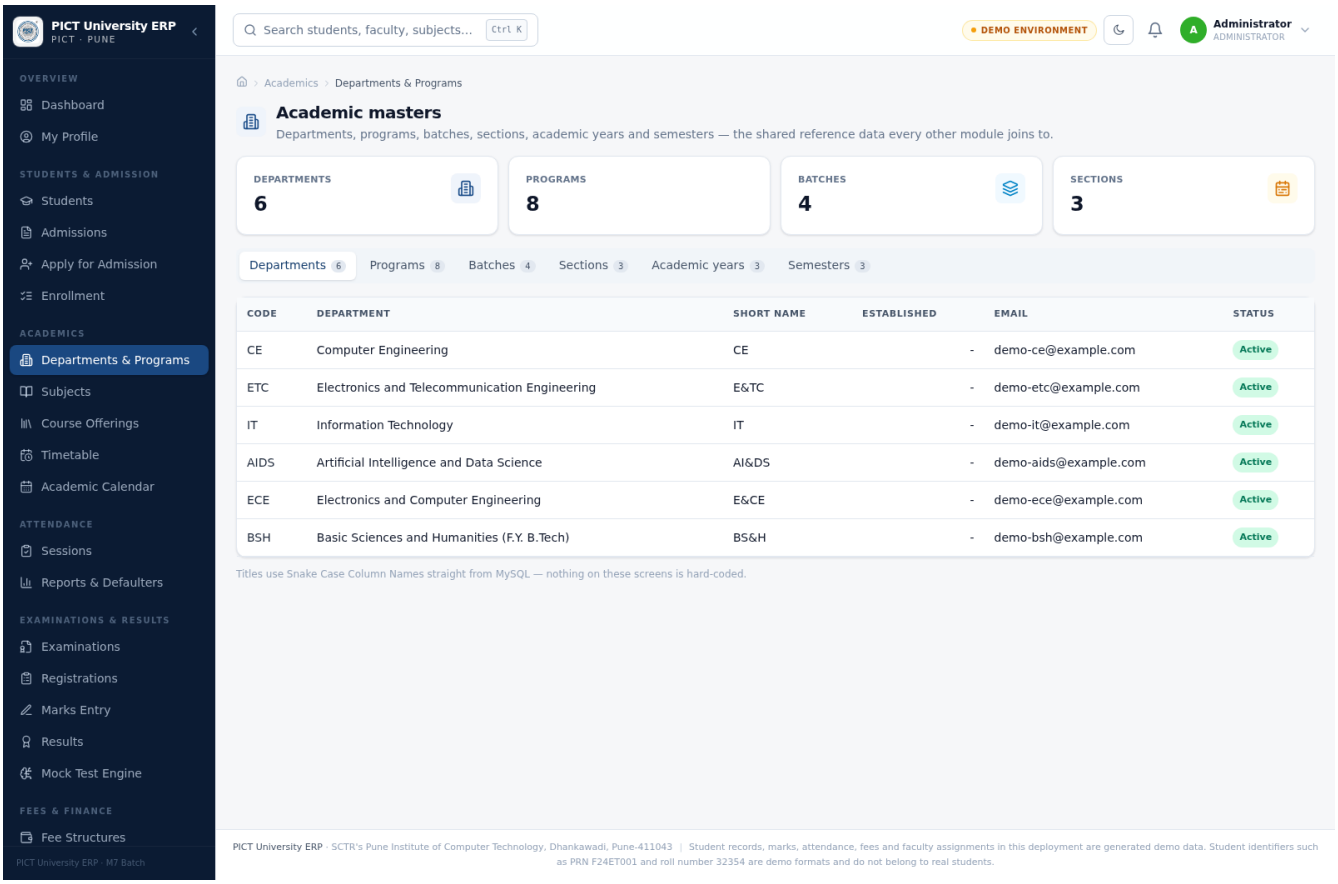


Figure 3. Academic masters page showing the shared department, program, batch and section structure. Email values use example.com demo placeholders; no genuine student/faculty identity is shown.

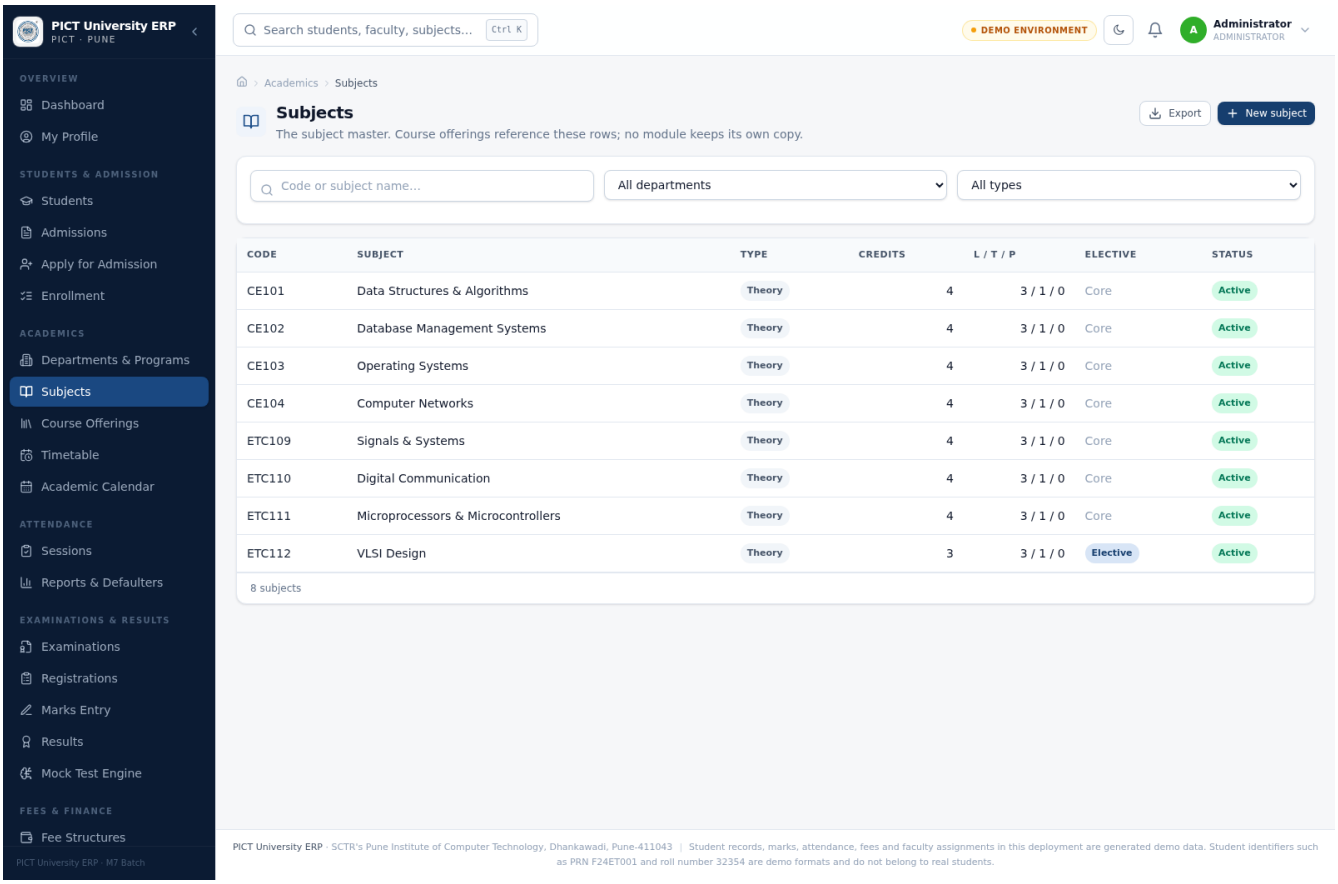


Figure 4. Subjects page with an illustrative subset of seeded subject codes and names; the page displays eight rows in this capture, not a fresh query of the 48-subject seed.

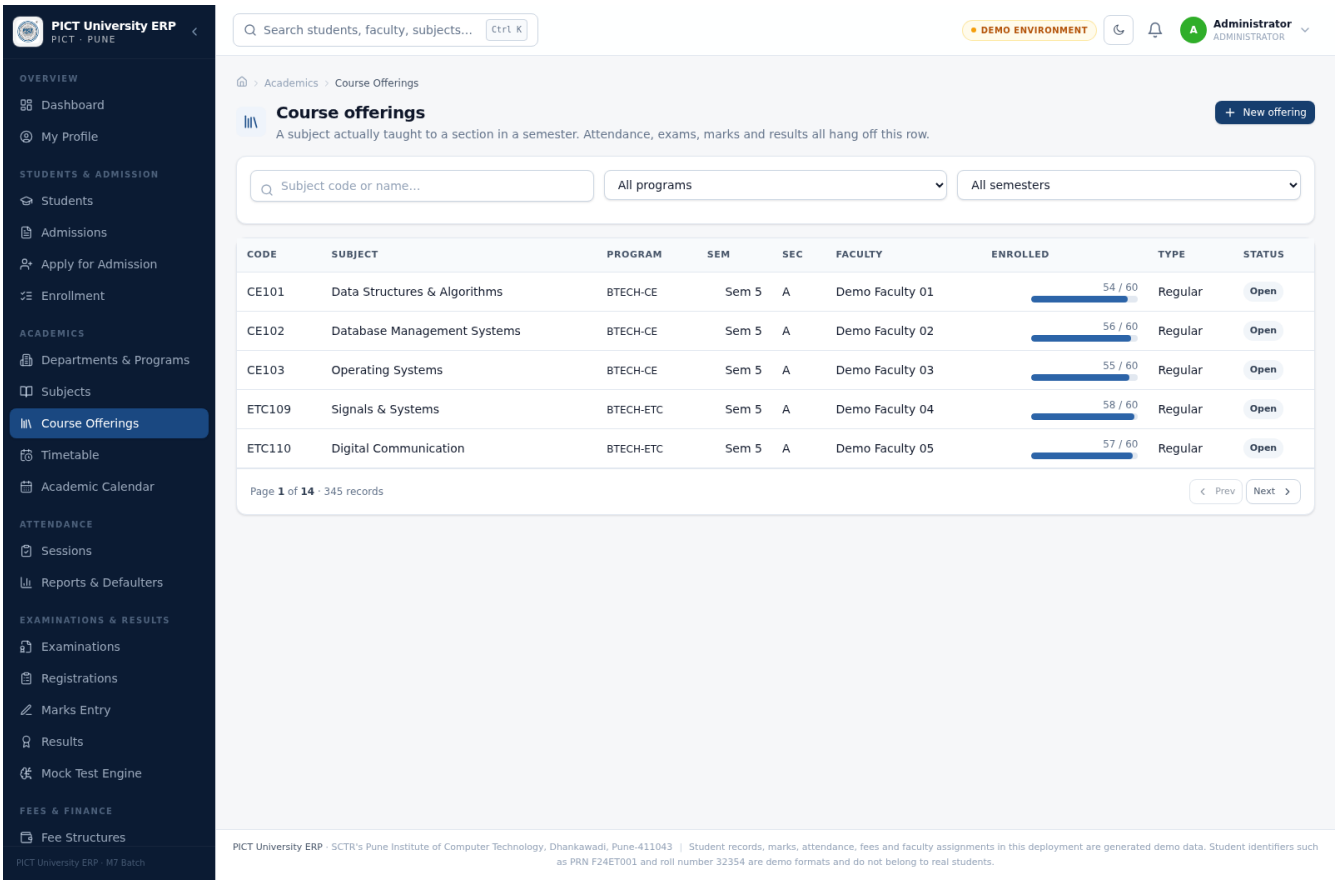


Figure 5. Course offerings page with five synthetic rows. Pagination metadata shows the repository-documented 345-offering seed scale, but the visible records and counts were supplied by the synthetic fixture.

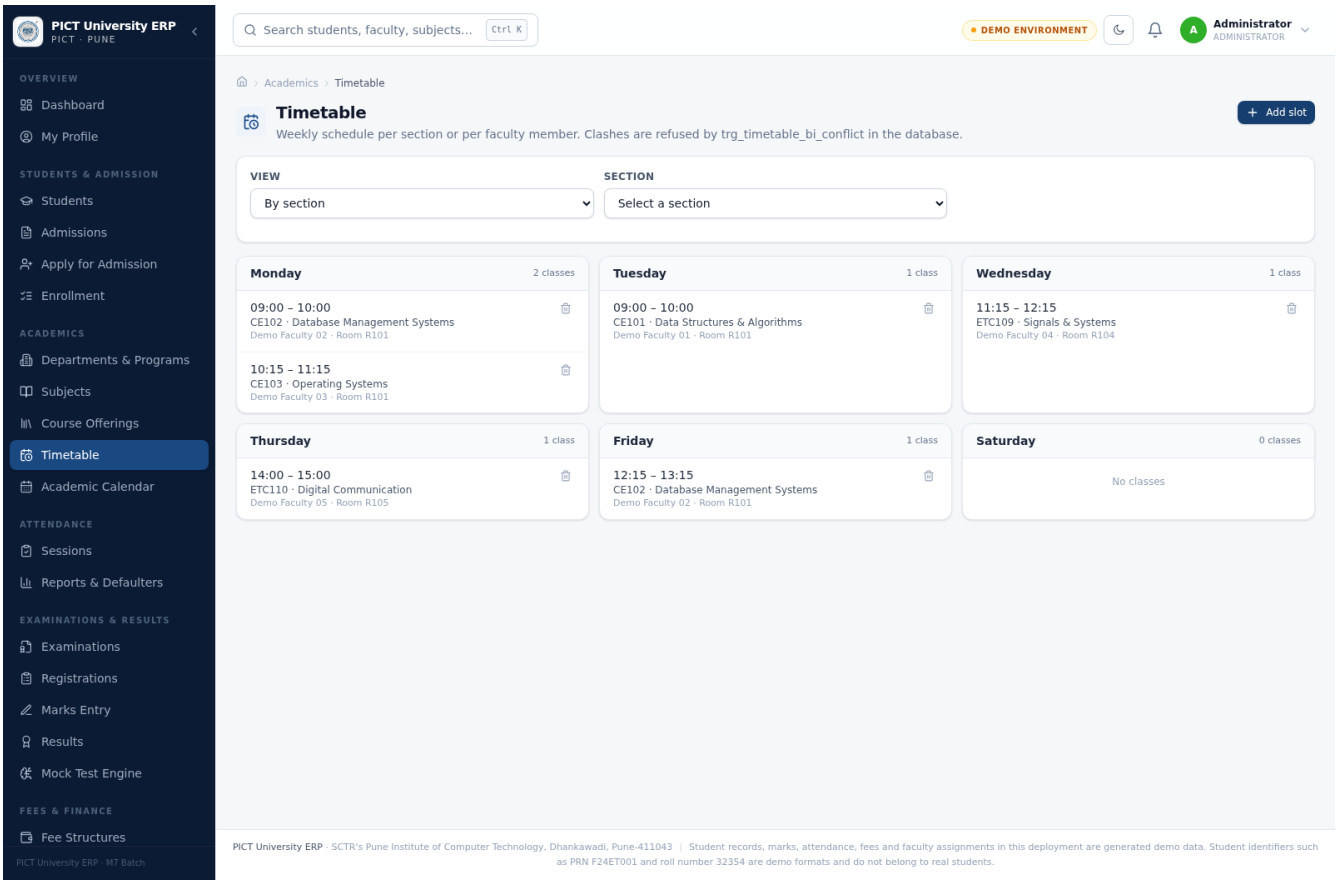


Figure 6. Weekly timetable cards with “Demo Faculty” labels and sample rooms/slots. No faculty member or live timetable record was queried.

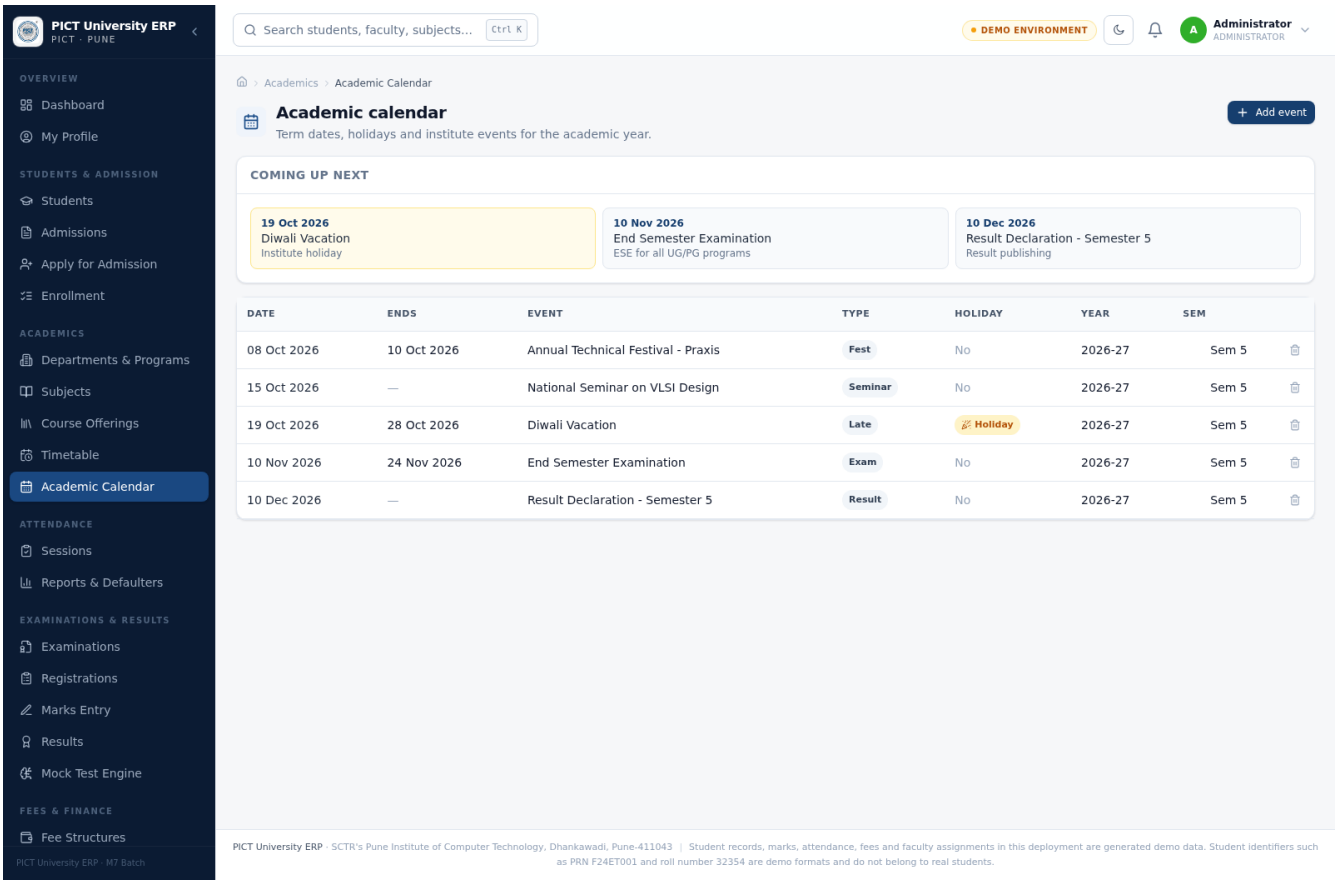


Figure 7. Academic calendar page. Event titles/dates reflect repository seed examples for illustration, but the rendered API response was intercepted and is not a live calendar query.

11. Cross-Module Integration

Partner module	Group 2 integration point	Boundary
Student & Admission	Admission assigns department/program/batch/section; enrolments reference students.	Student identity and approval belong to Module 1.
Faculty	Offerings and timetable reference faculty_id; faculty_subjects records subject allocations.	Faculty HR records and payroll belong to Module 7.
Attendance	Class sessions attach to course_offerings; attendance guard expects student enrolment for the offering.	Session creation/attendance marking belong to Module 3.
Exams & Results	Exam registrations reference course_offerings; the attendance eligibility logic reads offering enrolment/attendance.	Exam schedules, marks and results belong to Module 4.
Fees & Finance	Fee structures are associated with program and semester in the project integration documentation.	Billing/payment processing belongs to Module 5; do not infer a verified exam fee gate from the documentation alone.
Reports / Dashboard	Group 2 tables feed the current-term header, offering list, calendar and cross-module reporting.	The screenshots in this report use fixtures rather than live read models.

Cross-module verification issue: MODULE_INTEGRATION.md and README.md state that outstanding dues block exam registration. In the inspected database/05_procedures.sql implementation, sp_register_student_for_exam calculates v_dues, but the visible eligibility branch checks attendance and the insert path does not test v_dues > 0. The inspected trg_exam_reg_bi_eligibility also validates enrolment and attendance, not fee dues. Treat the documented fee gate as unverified until the procedure/trigger is corrected and exercised against MariaDB/MySQL.

12. Testing and Evidence Status

Evidence item	Status for this report
Repository/source review	Performed: academic UI pages, route file, service, table dictionary, seed fragments, procedure and trigger definitions reviewed.
React UI browser rendering	Performed with Playwright at 1600×1050 viewport for dashboard and five Group 2 pages; actual React components rendered without a captured page exception.
API/backend request	Not performed: backend API was unavailable; requests were intercepted in the browser.
MySQL/MariaDB integration	Not performed: database binaries/service and port 3306 were unavailable.
CRUD, procedure or trigger execution	Not performed in a live database; static code inspection only.
Project test suites	Not freshly run during report preparation. Repository summaries are recorded below with their discrepancies.

12.1 Existing repository test summaries (not this run)

Source document	Reported result	Qualification
README.md	10/10 database consistency checks and 158 API smoke checks reported as passing.	README also notes three MariaDB collation-related SQL rule statements remain under investigation.
PROJECT_REPORT.md §31	68 database rule tests, 158 API smoke tests and 40 workflow tests reported as passing.	The database-test count differs from the README summary; this report did not rerun either suite.

For a defense or release, capture a fresh test run from the current checkout and target database, keep stdout logs, and reconcile the conflicting database-test counts before presenting them as one result.

13. Limitations and Recommendations

1. Complete the generated Module 2 ER diagram: add batches, faculty_subjects and academic_calendar to scripts/generate_er_diagrams.py, update Mermaid source, and regenerate docs/er/. The present report calls out the omission rather than disguising it.
2. Reconcile the exam-fee gate: either add a server/database-backed dues check to the registration path and test it, or correct documentation and UI claims that imply the gate is enforced.
3. Run database rule, API smoke and workflow suites against the supported MariaDB/MySQL versions; align README and PROJECT_REPORT test counts.
4. Add explicit positive/negative tests for duplicate subject/program keys, bulk-enrolment rollback, at-capacity offerings, student/offering mismatch and timetable same-start conflicts.
5. If true interval-overlap prevention is required, extend the timetable trigger/constraints to compare start/end intervals, not only identical start_time values.
6. Regenerate browser captures after a DB-backed deployment and replace demo metrics with a clearly identified approved dataset; preserve the real official logo asset and canonical E&TC display name.

Other project-level checks, including the database dictionary counts and integrations outside Group 2, should be independently verified before a production or institutional deployment. This report does not claim production readiness.

14. Conclusion

Group 2 provides the academic backbone that connects curriculum definition to actual delivery. Its relational model separates masters, curriculum mapping, offerings, enrolments and timetable slots, while database-side guards protect the most consequential capacity and scheduling rules. The React interface presents these tasks within the shared ERP shell and exposes the academic calendar to the rest of the institution.

The repository contains substantial DBMS artifacts and working UI routes, but evidence must be stated precisely: screenshots here are React UI rendered with illustrative synthetic responses; stored routines and constraints were inspected but not executed in this environment. The diagram inventory and cross-module fee-gate discrepancy are explicitly documented so they can be resolved before a viva or final submission.

Appendix A. Recommended Demonstration Test Matrix

Case	Action	Expected outcome	Execution status
A1	Create two subjects with the same subject_code.	Second insert is rejected by the unique constraint and API surfaces a useful error.	Recommended; not run here.
A2	Enroll one student twice in one offering.	Second row is refused by uq_enrollments_student_offering.	Recommended; not run here.
A3	Bulk-enrol a cohort into a full offering.	Procedure rolls back and returns a capacity error; counters remain accurate.	Recommended; not run here.
A4	Insert a slot with end_time earlier than start_time.	trg_timetable_bi_conflict raises a readable error.	Recommended; not run here.
A5	Insert a duplicate faculty/room/section day + start slot.	Trigger and unique key refuse the duplicate.	Recommended; not run here.
A6	Attempt an ADMIN-only write as STUDENT.	API returns 403/unauthorized without changing database state.	Recommended; not run here.
A7	Mark attendance for a student not enrolled in the session offering.	Attendance trigger rejects the row.	Recommended; not run here.
A8	Register for an exam with low attendance and with outstanding dues.	Attendance case should be rejected; dues case must be separately fixed/verified before claiming a fee gate.	Recommended; not run here.
A9	Add a calendar event with a non-existent academic year or semester.	Foreign-key validation rejects the invalid association.	Recommended; not run here.

Appendix B. Repository Evidence and References

- README.md — project scope, stack, seed-count summaries, feature tour, API conventions, setup and test notes.
- PROJECT_REPORT.md — overall project report; especially §§7–9 (stack/architecture), §21 (Group 2), §31–32 (testing/setup).
- docs/MODULE_INTEGRATION.md — module reference, shared masters, cross-module flows and test/consistency notes.
- docs/DATA_DICTIONARY.md — schema inventory, Group 2 table definitions, indexes and foreign-key relationships.
- docs/ER_DIAGRAMS.md; docs/er/er_module2_academic_course.svg; docs/er/er_module2_academic_course.mmd — generated ER artifacts and recorded entity/relationship counts.
- database/02_tables.sql; database/03_constraints.sql — table definitions, foreign keys, indexes, unique keys and checks.
- database/04_seed.sql — academic year/semester seed, programs, subjects and calendar event examples. It is repository seed data, not a current database query.
- database/05_procedures.sql — sp_bulk_enroll_students and exam-registration implementation.
- database/06_triggers.sql — enrolment counters/guards, timetable conflict guard and exam eligibility trigger.
- backend/src/routes/academic.routes.ts; backend/src/controllers/academic.controller.ts; backend/src/services/academic.service.ts; backend/src/repositories/academic.repository.ts — Group 2 API and business/data-access path.
- frontend/src/pages/academics/AcademicsMastersPage.tsx, SubjectsPage.tsx, OfferingsPage.tsx, TimetablePage.tsx, CalendarPage.tsx — rendered module pages.
- frontend/src/pages/DashboardPage.tsx; frontend/src/components/dashboard/; frontend/public/brand/pict-logo-256.png — shared dashboard and available official PICT logo.
- docs/reports/group2/capture_screenshots.cjs — browser-capture script with synthetic API fixtures used for the figures; docs/reports/group2/screenshots/ — standalone screenshot files.

Related deliverable

The separate Group 2 Academic & Course study guide is stored under docs/viva_guides/. This formal project report is a distinct deliverable and does not replace the existing study guide.